

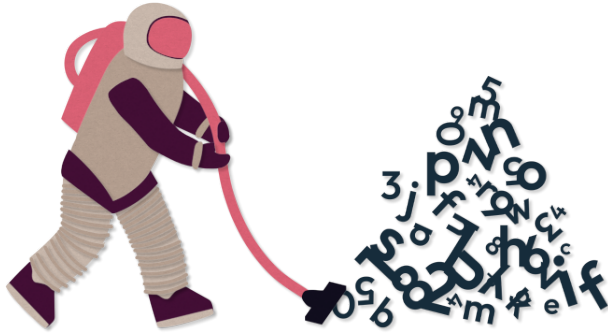
OBJECT-ORIENTED PROGRAMMING IN PYTHON



**Jumping
Rivers**

CREATING CLARITY WITH YOUR DATA.

Saving organisations like yours thousands of work hours.



Jumping Rivers is an analytics company whose passion is data and machine learning. We help our clients move from data storage to data insights. Our trainers and consultants come with over 100 years combined experience in R, Python, Stan, Scala and other programming languages.

Consultancy

- Posit (formerly RStudio) certified - the only UK partner.
- We have plans for managing all Posit products. From on-demand support to full care packages.
- As a Posit reseller, we can provide free consultancy.
- Experts in both data science and data engineering.
- Creation of bespoke dashboards and Shiny apps.
- Optimisation of your algorithms.



Training

- R/Python/Stan/Scala
- The Tidyverse
- Machine Learning and Tensorflow
- Dashboards with Shiny
- Statistical Modelling
- And many more!



hello@jumpingrivers.com

@jumping_uk

Introduction to Object-Oriented Programming

Python¹ is a fantastic general purpose programming language that has gained popularity with data analysts and statisticians over the last few years.

Numerous Python packages have been developed that make it easy to read and interact with data from various sources, to perform statistical analyses and to generate data visualisations. Those packages can be very large. The pandas package, which is the basis of a lot of data science work, contains around 400,000 lines of Python code as of 2023.

Having well-structured code is really beneficial when working in a software project, especially as that project becomes more complex. One way to achieve this is by using object-oriented programming (OOP).



¹ This course and all associated material were written using Python 3.10.



1.1 What is object-oriented programming?

OOP is a programming paradigm which provides a structure to programs such that properties and behaviours are bundled into individual objects. For example an object might represent a person. A person might have properties such as a name, an age etc., and behaviour like walking and talking. OOP is a means for modelling relevant entities as software objects² which have data associated with them and perform certain functions.

In Python, everything is an object.³ So if you have programmed using Python, you have been programming *with* objects.

In OOP, you design the objects first. Thinking, for example, what characteristics (data, behaviour) should all of the people (or fruit flies, or cars, or datasets) in my program exhibit and how should they interact with the other objects in that program.

² These might be real-world entities, like a person, or more abstract entities, like the files on a computer or a dataset.

³ Things like strings, but functions and even modules are objects.

1.2 Comparison to procedural programming

We'll see what it means to build a program using OOP over the coming chapters. But it might be useful to see how an OOP data science workflow differs from a more procedural approach first.

Let's say we have two datasets. One contains data about actors, and one about movies. For each of these datasets, we would need to read in the data and clean it up.

In the procedural version of the code we need a separate function

(with a separate name) for each type of dataset (`clean_actors()` , `clean_movies()`) and we need to remember how to read in the raw dataset (is the data tab- or comma-separated?).

```
import pandas as pd

# Note: Function definitions are hidden
actor_file = "actors-2023.csv"
movies_file = "movies-2023.tsv"

# Import the data
actors_raw = pd.read_csv(actor_file)
actors_clean = clean_actors(actors_raw)

movies_raw = pd.read_csv(movies_file, sep = "\t")
movies_clean = clean_movies(movies_raw)

# Do some analysis
analyse(actors_clean, movies_clean)
```

Suppose we were to rewrite the above, with a more object-oriented mindset. What are the main objects that are in play? And for those objects, what information do we need to store, and what behaviours can we identify?

Arguably, the most important objects are the datasets. Each one is stored on disk and imported, so we need to:

- know where it is stored on disk (information)
- know how to read it into memory (behaviour),
- know how to clean it up (behaviour)
- and (once imported), we need access to the data itself (information).

A rewrite of the above, that is slightly more object-oriented, might be as follows.

```
actor_file = "actors-2023.csv"
movies_file = "movies-2023.tsv"

# Import the data
actors = ActorDataset(actor_file)
actors.read()
actors.clean()

movies = MoviesDataset(movies_file)
movies.read()
movies.clean()

# Do some analysis
analyse(actors.data, movies.data)
```

This might seem a bit magical at the moment (and much of the detail has been hidden and will be revealed throughout the course).

Hopefully, the code hints that `actors` is a variable that knows how

to read (`actors.read()`) and clean up (`actors.clean()`) it's own dataset, and that we can subsequently access that data (`actors.data`).

Similarly, `movies` knows how to read in and clean up it's own data. The implementation of those behaviours may differ between an `ActorDataset` and a `MovieDataset` , but we can use the same names (`*.read()` , `*.clean()`) to call the relevant code.

Objects, Classes and Instances

Object-oriented programming is an approach to software design that is built upon modelling. Rather than thinking and coding procedurally (*this* should happen, then *that* should happen, and so on), you focus on the “things” and the different types of “thing” inside your program, what they represent and how they should interact.

If we are analysing a collection of cars, then we should try to identify those aspects of a car that we need to know about, need access to or need to replicate inside our program. With this, we build up an abstraction of “the car” that is useful to our program. But how do we encode that model so that our program can understand it, and how do we use our model of car-ness to create a representation of a specific car?

In this chapter we explain the mechanics of OOP in Python and introduce some concepts that are common to OOP in many other programming languages. Many Python packages are implemented using the approaches that are presented here, so you should gain a deeper understanding of how those packages work from following this chapter.

2.1 *The class is a blueprint*

If you are building a house, you need a way to specify what that house is going to look like and how it is going to be constructed. Similarly, if you want to represent houses in an object-oriented program, you need a way to specify how to make house objects. In OOP, our equivalent of the architect’s blueprint is the *class*.

A class is a formal declaration. Within a class, you specify what information will be stored in an object and what behaviours that object will have.

To define a class in Python, you use the `class` keyword. The simplest class you can define in Python is as follows:

```
class Empty():  
    pass
```

In the above, we have created a class called `Empty`. But a class is just a blueprint, so how do we create an object using that class?

2.2 *The object is an instance (of a class)*

In previous Python courses, or as part of your data science work, you may have used code similar to this:

```
import pandas as pd

my_data = pd.DataFrame(data={
    "heights": [1.56, 1.76, 1.44],
    "weights": [65, 68, 58]
})

my_data
#>   heights  weights
#> 0    1.56     65
#> 1    1.76     68
#> 2    1.44     58
```

Here, we have created a pandas `DataFrame` object and stored it in the variable `my_data`. The `type()` function can tell us what kind of object is stored in a variable:

```
type("abc")
#> <class 'str'>
type(1)
#> <class 'int'>
type(my_data)
#> <class 'pandas.core.frame.DataFrame'>
```

So, the `my_data` object is of class `pandas.core.frame.DataFrame`. In fact, if you look in the pandas source code, you will find a line of code that looks like:

```
class DataFrame(NDFrame, OpsMixin):
    # ... lots of code ...
```

The `DataFrame` class is a blueprint for `DataFrame` objects.¹ But it *also* provides a mechanism to construct `DataFrame` objects.

From our earlier example, we can therefore create an `Empty` object using the `Empty` class:

```
my_object = Empty()
type(my_object)
#> <class '__main__.Empty'>
```

We say that:

- `my_object` is an instance of the class `Empty`; or
- `my_object` is an `Empty` (object).

In Python, everything is an object. We can check if a given object is an instance of a particular class using the function `isinstance()`:

```
isinstance(my_data, pd.DataFrame)
#> True
isinstance(my_object, Empty)
#> True
isinstance(my_object, pd.DataFrame)
#> False
```

¹ Note that class names are typically written in `UpperCamelCase` - capitalized words that are collapsed together. This is standard practice in Python.

2.3 Attributes store information

Attributes are values that are associated with an object.² If we were to define a class for cars, the values associated with a given car might store the model name or the fuel efficiency. Here, we store these as the `.model` and `.mpg` attributes:

```
class Car():
    def __init__(self, model, mpg):
        self.model = model
        self.mpg = mpg
```

There is some new syntax to note here:

- `__init__()` - this is the object *constructor* for the class. It tells us how we create, or initialise, a new instance of the `Car` class;
- `self` - this variable is a reference to the current instance of the `Car` class;
- `self.model = ...` and `self.mpg = ...` - this is how we attach attributes to the current instance of the `Car` class.

We can use the `Car` class to create two new instances.

```
mazda = Car(model="Mazda RX4", mpg=21.0)
firebird = Car(model="Pontiac Firebird", mpg=19.2)
```

Then, we can access the attributes from these `Car` instances:

```
mazda.model
#> 'Mazda RX4'
firebird.mpg
#> 19.2
```

This syntax, for accessing attributes on an object, should be familiar if you have worked with numpy:

```
import numpy as np
tree_heights = np.array([10.5, 2.6, 7.8])

# Accessing the `dtype` attribute for the array
tree_heights.dtype
#> dtype('float64')
```

2.4 Methods enable behaviour

Methods are functions that are bound to an object. Even if that sounds unfamiliar, you may have used code that employs this idea. In the following we are calling the `.min()` and `.mean()` *methods* on the numpy array `tree_heights`:

```
tree_heights.min()
#> np.float64(2.6)
tree_heights.mean()
#> np.float64(6.9666666666666666)
```

Methods are defined similarly to functions, but they are defined inside

² You may also find attributes being referred to as “fields” or “properties”.

the class definition. The `__init__()` constructor (discussed above) is a method. A `self` parameter in the method signature allows the method to refer to the current instance:

```
class Artist():
    def __init__(self, artist, paintings):
        self.artist = artist
        self.paintings = set(paintings)

    def has_painted(self, painting):
        return painting in self.paintings

pablo = Artist("Picasso", {"Guernica", "La Vie"})
pablo.has_painted("Mona Lisa")
#> False
```

2.5 OOP key idea: abstraction

So you have seen how to define classes, and how to use those classes to create objects. Classes are the foundation for the implementation of OOP in Python (and we will study classes in more detail in subsequent chapters).

OOP is a software design methodology. Four key ideas underpin this approach: abstraction, encapsulation, inheritance and polymorphism. We will illustrate each of these ideas when we introduce them.

The ideas in this chapter relate to abstraction. This is a process where you model some collection of entities (often real-world entities, like the cars discussed here) and identify any information or behaviours that those entities exhibit that you need to represent in code. Some cars are red, and some are blue, but if the program you are writing doesn't need to know, then there's no reason to store the colour of a car in your `Car` class.

Defining Classes - Instance Methods and Attributes

In the previous chapter we saw how OOP will lead you to identify the most important kinds of object in your program, and make a computational model of each kind of object using classes. In Python, any object-associated behaviour and information are encoded as methods and attributes, respectively.

Here, we study methods and attributes in more detail. Our focus will be on class *instances*¹, so we will study *instance methods* and *instance attributes*. There are also class methods / attributes and static methods. These are attached to the class *itself*, not to a particular instance, and are studied later in the course.

¹ a particular car or tree or dataset, rather than the blueprint for one.

While working through the chapter, we will define several classes. Think about these classes from the user's perspective. Ideally, the user shouldn't need to know anything about the internals of a class, including:

- how the methods work;
- how the attributes are stored inside the class.

This idea of *encapsulation* is important throughout OOP and will be emphasised in this chapter.

3.1 The Cohort class

We will consider how to store results from a study carried out at different locations around the country as our main example in this chapter. The data collected are:

- The heights and weights of individuals;
- The location of the centre where the study took place.

Following the modelling discussion in the previous chapter, we might create a `Cohort` class to capture this information. Methods and attributes defined on the class could be used to work with a specific `Cohort` instance (for example, the data from the Newcastle study cohort).

For our `Cohort` class, how should we store the heights and weights of the individuals? There are many choices:

- a `list` of (height, weight) tuples;
- two separate lists (one for heights, one for weights);
- a `numpy ndarray`; or
- a `pandas DataFrame`.

The user doesn't care and shouldn't need to know. They just want to be able to create an object of this type and use it. That is, we want to separate the interface of the class (how the user interacts with the class and its instances) from the implementation of that class (how the data is stored inside, and manipulated by the class and its instances).

We could begin to define our `Cohort` class in Python as follows.

```
import pandas as pd

class Cohort():
    def __init__(self, heights, weights, centre):
        self.data = pd.DataFrame({
            "heights": heights,
            "weights": weights
        })
        self.centre = centre
```

Important things to reiterate here are:

- `class Cohort:` - we are defining a new class (or object blueprint) called `Cohort`;
- `__init__()` - the constructor. This tells us how we create, or initialise, a new instance of the `Cohort` class;
- `self` - a reference to the current instance (discussed below);²
- `self.data = ...` and `self.centre = ...` - defines how the `data` and `centre` attributes for an instance will be initialised.

² `self` is a Python keyword. The other parameters in the constructor behave just like function parameters.

Once our class is defined, with a constructor³, we can create new instances of our object.

```
heights = [1.4, 1.47, 1.47, 1.58, 1.25]
weights = [32, 39, 33, 66, 24]
centre = 'NCL'
my_cohort = Cohort(heights, weights, centre)
```

³ Strictly we don't need `__init__()` if there is no initialisation required for the object.

3.2 Instance methods

Instances of a class have access to their own attributes and methods. When defining these *instance methods*, we use the `self` keyword in the parameter list. This keyword allows the method body to refer to the current instance of the class (and its attributes and methods).

```
class Person():
    def __init__(self, name):
        self.name = name

    def speak(self):
        message = "Hello, I'm " + self.name + "."
        print(message)

charlie = Person("Charlie")
lucy = Person("Lucy")
```

```
charlie.speak()
#> Hello, I'm Charlie.
lucy.speak()
#> Hello, I'm Lucy.
```

Suppose we want to add the ability for our `Cohort` instances to generate a scatter plot of their data (for example, heights against weights). This is a perfect opportunity to add an instance method to our `Cohort` class definition.

```
import matplotlib.pyplot as plt

class Cohort():
    # ...
    def plot(self):
        pandas_data = self.data
        scatter_plot = pandas_data.plot.scatter(
            x="weights", y="heights", title=self.centre
        )
        plt.show(scatter_plot)
```

Once we have defined our method we can make use of it.

```
my_cohort = Cohort(heights, weights, centre)
my_cohort.plot()
```

The `self` argument is passed implicitly when we call `*.plot()`. It refers to the object calling the method, which in this example is `my_cohort`. This allows the method to access the attributes of the specific instance that we have created.

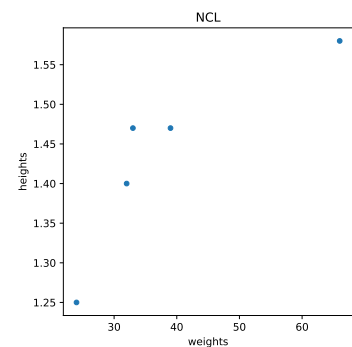


Figure 3.1: A simple scatter plot generated via the 'plot()' method.

Returning a value from an instance method

Methods are comparable to functions and everything you have learned about python functions applies to methods as well. In particular, the `return` statement works identically inside functions and methods: it sends results back to the caller.

As an example, we could define a method on our `Cohort` class to return the maximum height in the dataset.

```
class Cohort():
    # ...
    def max_height(self):
        # Since self.data is a pandas DataFrame,
        # we can use the pandas `*.max()` method.
        return self.data["heights"].max()
```

```
heights
#> [1.4, 1.47, 1.47, 1.58, 1.25]

my_cohort = Cohort(heights, weights, centre)
my_cohort.max_height()
```

```
#> np.float64(1.58)
```

Our implementation of `*.max_height()` works because the height data is stored inside a pandas `DataFrame`. If we chose to change how the height data was stored, we would have to rewrite the `*.max_height()` method accordingly.

Setting an attribute using an instance method

In the above, the class constructor (`*.__init__()`) was used to initialise the attributes attached to an object (`self.data = ...`, `self.centre = ...`). Attributes can also be modified from other instance methods.

Suppose we defined a class to hold a dataset. We might need a method to read the data into memory from disk. By assigning the data to an attribute, that method can attach the data to its instance.

```
import pandas as pd

class MovieDataset():
    def __init__(self, filepath):
        self.filepath = filepath
        self.raw_data = None

    def read(self):
        self.raw_data = pd.read_csv(self.filepath)
```

That methods can modify instance attributes may not be too surprising. Indeed, we have been setting attributes in our constructor method (`*.__init__()`) and this is an instance method (because its `self` argument gives it access to the current instance).

3.3 *Instance attributes*

In fact, it is possible to modify instance attributes from outside of the class definition.

```
movies = MovieDataset("abc.csv")

movies.filepath
#> 'abc.csv'

movies.filepath = "def.csv"
movies.filepath
#> 'def.csv'
```

As a class author, you might not want your users to be able to modify a particular attribute in this way. Unlike many other languages, Python doesn't have a way to specify that an attribute (or method) is *private*⁴. Instead, there is a convention that attributes and methods that have an underscore prefix should be treated as if they are private.

⁴ By private we mean something that is associated with an instance that no other object should have access to.

```
class TopSecret():
    def __init__(self, name, secret):
        self.name = name # feel free to access this
        self._secret = secret # please don't access this

a_secret = TopSecret(
    name="Cooking rice",
    secret="Boil for 10, fluff, steam for 10"
)
```

We can access the name of this secret:

```
a_secret.name
#> 'Cooking rice'
```

We can even access the secret attribute, but perhaps we shouldn't because it is marked as 'hidden'

```
a_secret._secret
#> 'Boil for 10, fluff, steam for 10'
```

Because it is 'hidden' (by pythonic convention), the class author has freedom to reimplement that attribute. Maybe they will encrypt these really important secrets in a future version of the class.

3.4 Interlude: Mutability

We have seen how attributes can be set from inside (using `self.attribute = ...` in methods) and outside (using `object.attribute = ...` assignment) the class. Because objects can be modified in this way, we describe them as *mutable*. This ability to change the values associated with an object after its initial construction can, however, lead to some headaches.

Suppose we have a `Person` object referenced by the variable `marcie`. This object has one attribute (`*.name`).

```
marcie = Person("Marcie")
marcie.name
#> 'Marcie'
```

We can make a second variable that references the same object, and see that it has the same attribute value as the original variable.

```
marcie2 = marcie
marcie2.name
#> 'Marcie'
```

Now if we modify the `name` attribute for the second variable, we find that a corresponding change has occurred in the first variable.

```
marcie2.name = "Lucy"

marcie.name
#> 'Lucy'
marcie2.name
```

```
#> 'Lucy'
```

This happens because the two variables point to the same object. Any modification to that object will impact on both variables and on wherever they are subsequently used in the program. As such, if a function (or method) has a side-effect where it mutates a user-supplied object, that function can make your code hard to reason about.

Data Classes and Immutability

“Data Classes” provide a streamlined way to define classes and they can alleviate the problems of mutability. They are a nice way to model entities whose data do not change, which is quite a common setting in data analysis work.

To define a data class you need to import the `dataclass` decorator.⁵ When defining the attributes of a data class, you must specify the *type* of data that should be stored in that attribute.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Character():
    name: str
    age: int
```

This defines a new class `Character`, where each instance has a `name` and an `age` attribute. These attributes should be a string (`str`) and an integer (`int`), respectively.

We can use the `Character` class, much like any other class.

```
sally = Character(name="Sally", age=10)
sally
#> Character(name='Sally', age=10)
```

Because we defined this data class using `frozen=True`, the class does not allow mutation. So if we attempt to change an attribute, Python will complain.

```
sally.name = "Lucy"
#> dataclasses.FrozenInstanceError: cannot assign to field
#> 'name'
```

```
sally
#> Character(name='Sally', age=10)
```

The constructor for data classes is added implicitly by Python (by default). For our example, the constructor would be equivalent to the following (but with type-checking on the `name` and `age` arguments):

```
class Character():
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

⁵ A decorator is a function that extends the behaviour of an object without directly modifying it. Here, the decorator is telling Python to extend the `Character` class with the properties of a `dataclass`. We will use decorators elsewhere in the course. But how they work and how to define them are out of the scope of this course.

3.5 *OOP key idea:* encapsulation

Ultimately *encapsulation* is about two things:

- keeping things that belong together *together*; and
- hiding things you don't want others to see.

Classes, and the objects constructed from them, give us a sensible way to hold different bits of related data, and methods that make sense for that data, in one place. This can simplify your programming life. It's easier to find the function you need if it is attached to the object you are working with.

But some of the data and methods associated with an object may only be for use by that object. A user may find it difficult to decide which of the 10 methods and 12 attributes associated with an object they've constructed are correct for their current need. Hence it is useful, when defining classes, to hide any methods and attributes which are not for public use.

In some ways, the value of implementation hiding shouldn't be a new concept. A user doesn't need to know how a function works to be able to make use of it. Nor do I need to know how my phone works to make a phone call.

Object-Oriented Design - Inheritance

Inheritance is a fundamental OOP concept. It provides a way to model the relationship between similar types of object and can dramatically reduce the amount of code you need to write in your programs.

To solidify our understanding of inheritance, we will do a bit more modelling.

You are analysing the entire universe. There are many entities out there. Galaxies, stars, planets, moons, spacecraft and a tiny, but significant, amount of life.

Let's just stick to stars and planets for now. These are different types of objects. So we might consider representing them using two separate classes. But, they have many properties in common: mass, radius, temperature. So in our classes, we might need to include these common properties as attributes. Depending on our application, there may be properties of stars that are not relevant to planets, and vice versa. Maybe we wouldn't even consider asking whether a star has water, or what the luminosity of a planet is.

With this in mind, we might end up with the following two classes:

```
class Star():
    def __init__(
        self, name, mass, radius, luminosity
    ):
        self.name = name
        self.mass = mass    # kg
        self.radius = radius # km
        self.luminosity = luminosity # W
```

```
class Planet():
    def __init__(
        self, name, mass, radius, has_water
    ):
        self.name = name
        self.mass = mass    # kg
        self.radius = radius # km
        self.has_water = has_water # True / False
```

```
earth = Planet("Earth", 5.972e24, 6371, True)
sun = Star("The Sun", 1.989e30, 696340, 3.828e26)
```

4.1 Subclasses and the class hierarchy

The two classes seem pretty straightforward and we could add a `Moon` class in a similar manner. But, in these two classes, we already have a bit of duplication: the code for defining the ‘name’, ‘mass’, and ‘radius’ of an instance are identical for a `Star` and a `Planet`. Adding `class Moon` would lead to even further duplication.

We will use *inheritance* to simplify the code before making any further changes.

Inheritance is a modelling approach that revolves around “is a” questions. A dog is an animal and a cat is an animal, but a cat isn’t a dog.

Here we would ask:

- is a star always a planet? Nope
- is a planet always a star? No, again.
- is there some more general type of object that encompasses both Stars and Planets?

Well, yes:

- The Earth is a planet, which is a celestial object, and
- the Sun is a star, which is a celestial object;
- in fact, every planet and every star is a celestial object.

In OOP, we model these “is a” relationships using a *class hierarchy*.

If we had a `CelestialObject` class, then we could make these relationships explicit.

```
class CelestialObject():
    # ... details omitted ...
    pass

class Planet(CelestialObject):
    # ... details omitted ...
    pass

class Star(CelestialObject):
    # ... details omitted ...
    pass
```

The `class TheSubClass(TheSuperClass):` syntax tells Python that every instance of `TheSubClass` is also an instance of `TheSuperClass`.

So, in our example, since `earth` is a `Planet`, it is also a `CelestialObject` (and similarly for `Star` objects).

We say that:

- `Planet` is a *subclass* of the *superclass* `CelestialObject`
- or, that `Planet` inherits, extends or derives from `CelestialObject`.

4.2 The subclass constructor

How does our class hierarchy simplify our code, though?

The crux of this is that all the attributes and methods that are available

on a superclass are immediately available on its subclasses.

So, if we define a constructor on our superclass (`CelestialObject`) that sets up the 'name', 'mass', and 'radius' for an instance, we can use that constructor to set up these attributes during the construction of a subclass instance (`Planet` or `Star`) without duplicating that code.

```
class CelestialObject():
    def __init__(self, name, mass, radius):
        self.name = name
        self.mass = mass
        self.radius = radius

class Planet(CelestialObject):
    def __init__(
        self, name, mass, radius, has_water
    ):
        # Use the superclass constructor to set up common
        # attributes
        super().__init__(name, mass, radius)
        # Then add any planet-specific attributes
        self.has_water = has_water # True / False

class Star(CelestialObject):
    def __init__(
        self, name, mass, radius, luminosity
    ):
        super().__init__(name, mass, radius)
        self.luminosity = luminosity
```

Creating objects works exactly as it did when the constructor code was duplicated:

```
earth = Planet("Earth", 5.972e24, 6371, True)
sun = Star("The Sun", 1.989e30, 696340, 3.828e26)
```

Also, because we have extracted a more general class out from our `Planet` and `Star` classes, we can now represent objects that are neither planets nor stars:

```
enceladus = CelestialObject(
    "Enceladus", 1.080e20, 252.1
)
```

Reduced maintenance

Code duplication often leads to more maintenance. In our original implementation, if we made a small change to `Planet` then we may have to make the same change to `Star` so that the two classes are consistent with each other. Limiting the amount of code-duplication in a project is one of the main reasons why we write functions and define classes. Here we will show a typical maintenance task (changing the data type stored by the classes) and see how simply it can be performed now that our class hierar-

chy is in place.

At present, our classes store measurements as bare numbers. Even if we document that the input values should be in kilograms and kilometres:

- a user could create a `Planet` object with distances in astronomical units;
- and a function that uses our objects may mistakenly assume the distances are in metres.

Suppose we decided that rather than storing measurements as bare numbers we should store the measurement unit as well. This seems pretty wise.

In 1999 the “Mars Climate Orbiter” was lost when it should have entered orbit around Mars. Precisely where it ended up, we don’t know. But we do know that the orbiter was lost because of a software-level mix up on the choice of units. NASA’s software used metric units, but the builder’s used US customary units (similar to Imperial units).

The `measurement` package simplifies working with different unit systems:¹

```
from measurement.measures import Distance

earth_radius = Distance(km=6371)
earth_radius
#> Distance(km=6371.0)
earth_radius.british_yd
#> 6967422.403805775
```

Making the change is quite simple, because we’ve limited the number of places where weights and distances are handled:

```
from measurement.measures import Distance, Weight

class CelestialObject():
    def __init__(self, name, mass, radius):
        self.name = name
        self.mass = Weight(kg=mass)
        self.radius = Distance(km=radius)
```

Now any place where our `CelestialObject`s are used, the measurement unit is explicit.²

```
earth = Planet("Earth", 5.972e24, 6371, True)
earth.radius
#> Distance(km=6371.0)
earth.radius.british_yd
#> 6967422.403805775
```

¹ But still provides no guarantees that your orbiter will make it to Mars.

² If you redefine a class in an interactive session, these changes will not immediately affect any subclasses or pre-existing instances in that session. You will need to re-evaluate any code blocks that depend upon the redefined class.

4.3 Methods and inheritance

All of our `Planet` and `Star` instances have a `radius` attribute. So, assuming they are spherical, we could add a method to the two classes to compute approximate volumes. But the calculation would be identical.

We have seen that attributes that are defined on a superclass

(`CelestialObject`) are available to subclasses (`Planet`). Methods are also inherited by subclasses.

Let's put the `volume` method into `CelestialObject`:

```
from measurement.measures import Volume
import math

class CelestialObject():
    def __init__(self, name, mass, radius):
        self.name = name
        self.mass = Weight(kg=mass)
        self.radius = Distance(km=radius)

    def volume(self):
        # Approximate volume, assuming spherical
        r = self.radius.km
        return Volume(
            teralitre=(4/3) * math.pi * (r ** 3)
        )
```

Once the subclasses have been redefined the new method can be used to get the volumes of our objects:

```
enceladus = CelestialObject(
    "Enceladus", 1.080e20, 252.1
)
enceladus.volume()
#> Volume(Tl=67113076.3089104)

earth = Planet("Earth", 5.972e24, 6371, True)
earth.volume()
#> Volume(Tl=1083206916845.7535)
```

Users of the class might find using `*.volume()` a little strange though. The volume should really be a fixed entity for a given planet (like radius is). But we use attribute-syntax to access the radius (`*.radius`), and method-syntax (`*.volume()`) to access the volume. The user has to know which values are stored as data, and which are computed.³

³ Beyond attributes and methods, we can also define instance 'properties' which are computed, but can be accessed as if attributes. So they look like stored data, even though they aren't. This is beyond the scope of the course.

Overriding parent methods

All of the attributes and methods that are associated with a class are inherited by its subclasses. But, subclasses are free to define their own implementation of an inherited method.

For example, suppose we had a class `WeatherReport` that takes some weather data and makes a human-readable report from it.

```
class WeatherReport():
    def __init__(self, weather_data):
        self.chance_of_rain = weather_data["rain"] * 100
        self.cloud_coverage = weather_data["cloud"] * 100
```

```
def report(self):
    report_text = f"""Weather Report
-----
Chance of Rain: {self.chance_of_rain} %
Cloud Coverage: {self.cloud_coverage} %
"""
    return report_text
```

Note that we are quoting strings using three double-quotes `""" the string """` in the `report()` method. With these we can easily include newlines and other quote-characters within the string. We are also using the Python “f-string” syntax with which you can embed variables in strings: `f"The value is {value}"`.

We can create a report as follows:

```
weather_report = WeatherReport({
    "rain": 0.4,
    "cloud": 0.2
})
print(weather_report.report())
#> Weather Report
#> -----
#> Chance of Rain: 40.0 %
#> Cloud Coverage: 20.0 %
```

That would be fine, if we always wanted a plain text report. Suppose some users of our code would prefer to make a report for online presentation. The text would need to be wrapped in HTML markup.⁴

One solution may be to add an `OnlineWeatherReport` subclass. We want the `report()` method to return HTML so we have to override the parental `report` method.

```
class OnlineWeatherReport(WeatherReport):
    def __init__(self, weather_data):
        super().__init__(weather_data)

    def report(self):
        html = f"""<div class="weather-report">
            <h2>Weather Report</h2>
            <p>Chance of Rain: {self.chance_of_rain} %</p>
            <p>Cloud coverage: {self.cloud_coverage} %</p>
        </div>"""
        return html
```

```
online_report = OnlineWeatherReport({
    "rain": 0.4,
    "cloud": 0.2
})
print(online_report.report())
#> <div class="weather-report">
#>     <h2>Weather Report</h2>
```

⁴ Hypertext-markup language, HTML, powers the content of the internet. If you look at the “page source” for a website in your browser, the language you are looking at is HTML.

```
#>      <p>Chance of Rain: 40.0 %</p>
#>      <p>Cloud coverage: 20.0 %</p>
#>      </div>
```

4.4 Interlude: Magic methods

In Python, every class inherits from a base `object` class. We can examine the class hierarchy using the `mro()` method (on a class, not a class instance).⁵

```
class F():
    pass
```

```
class G(F):
    pass
```

We see from the output of `mro()`, that:

- `object` is the superclass for `F`
- `F` and `object` are superclasses for `G`

```
print(F.mro())
#> [<class '__main__.F'>, <class 'object'>]

print(G.mro())
#> [<class '__main__.G'>, <class '__main__.F'>, <class
#> 'object'>]
```

We can also find all the methods and attributes defined for a given object, using the `dir()` function.

```
f = F()
dir(f)
#> ['__class__', '__delattr__', '__dict__', '__dir__',
#> '__doc__', '__eq__', '__format__', '__ge__',
#> '__getattr__', '__getstate__', '__gt__', '__hash__',
#> '__init__', '__init_subclass__', '__le__', '__lt__',
#> '__module__', '__ne__', '__new__', '__reduce__',
#> '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
#> '__str__', '__subclasshook__', '__weakref__']
```

Despite there being no methods or attributes defined on the class `F`, there are many things in the list of available methods. These are inherited from `object`.

What’s with those funny names and all those underscores: `__eq__`, `__ge__`, `__init__`, `__str__`?

These are “magic methods”⁶ that are called implicitly by the class or by an instance. We have already seen `__init__` many times: when you call `Planet(arguments)` you create a `Planet` instance. You don’t actually call `__init__()`, but it is used when creating the object that is returned.

Similarly, `__eq__` and `__ge__` wouldn’t be called directly by a user, but they are used if the user attempts to compare two objects:

⁵ “MRO” stands for “Method Resolution Order”. When you call a method on an object, several classes may define that method. The implementation that is used comes from the first class in this list that defines that method.

⁶ Sometimes called “dunder methods”. “Dunder” being short for double underscore.

- `f == g` uses the `__eq__` method on `f`
- `f >= g` uses the `__ge__` method on `f`.

String-conversion and printing

Here we will look at the `__str__` magic method in depth. This is used when calling `print()` on an object.⁷

⁷ `str` here is short for string.

Our `Cohort` class is defined as follows (plotting code has been removed for clarity):

```
import pandas as pd

class Cohort():
    def __init__(self, heights, weights, centre):
        self.data = pd.DataFrame({
            "heights": heights,
            "weights": weights
        })
        self.centre = centre
```

As an example consider the output we currently see if we `print()` our `Cohort` instance.

```
heights = [1.4, 1.47, 1.47, 1.58, 1.25]
weights = [32, 39, 33, 66, 24]
centre = "NCL"

my_cohort = Cohort(heights, weights, centre)
print(my_cohort)
#> <__main__.Cohort object at 0x79d4243fd430>
```

This is not very useful. We can't see the data that is stored there, or any other information about the object, other than it's memory address. To add a nicer print method for our class, we need to implement a `__str__()` method.

```
class Cohort:
    def __init__(self, heights, weights, centre):
        self.data = pd.DataFrame({
            "heights": heights,
            "weights": weights
        })
        self.centre = centre

    def __str__(self):
        output = f"""Data from centre : {self.centre}

{self.data.__str__()}
"""
        return output
```

Any `__str__()` method should return a string. Here we use f-strings to

create the output string and make a call to the `__str__()` method for the internal `DataFrame` so that we don't have to reinvent the wheel for printing data frames.

Now if we print a `Cohort` object, a more sensible string is outputted to the console.

```
my_cohort = Cohort(heights, weights, centre)
print(my_cohort)
#> Data from centre : NCL
#>
#>      heights  weights
#> 0      1.40      32
#> 1      1.47      39
#> 2      1.47      33
#> 3      1.58      66
#> 4      1.25      24
#>
```

We should note that `__str__` is the method that gets called when we use `print()`. However this does not change the behaviour of the output when we just call the variable name. That is handled by `__repr__`.⁸ We can go ahead and define this method for our class too:

```
def __repr__(self):
    return f"""Cohort Instance:

    {self.__str__()}
    """
```

⁸ For a discussion on the differences between `__repr__` and `__str__` see <https://stackoverflow.com/questions/1436703/difference-between-str-and-repr>.

Other magic methods

There are a host of magic methods for adding behaviour to our own classes of objects. Each is documented in the Python documentation, but examples and organisation are lacking. A good reference we have found is <https://rszalski.github.io/magicmethods/>

4.5 OOP key idea: inheritance

Inheritance is both a modelling strategy and a way to structure OOP systems.

When modelling, we are thinking how to represent the information associated with (and the behaviours of) some collection of things in code. From this we may end up with a few different classes to represent the different types of thing in that collection. With inheritance, we now start to think about the relationships between those classes (and the types of thing that they represent). If we have a `Dog` class and a `Cat` class, we may end up implementing similar behaviours across the two classes. By implementing an `Animal` class, from which both `Dog` and `Cat` inherit, we can move the duplicated implementations into `Animal`. This may reduce future maintenance. A slight problem is that this ties the implementation of

`Dog` and `Cat` together: any time we make changes to `Animal`, we have to be certain that those changes are appropriate for all it's subclasses.

Defining Classes - Class and Static Methods

5.1 Composition: A has-a relationship

In this chapter we will learn about “class” and “static” methods. These are methods associated with a class that *do not* have access to an instance of that class. This is unlike “instance” methods, which all have access to a class instance via their `self` parameter.

To illustrate why you would need “class” and “static” methods, we will design a class for working with movie datasets:

```
import pandas as pd

class MoviesDataset():
    def __init__(self, raw):
        # 'raw' and 'cleaned' are pandas DataFrames
        self.raw = raw
        self.cleaned = self.clean()

    def clean(self):
        return self.raw.dropna()
```

To use this class, we pass in a `pandas DataFrame`.

```
import numpy as np

raw_movies = pd.DataFrame({
    "film": ["Toy Story", "Up", "WALL-E", "Elemental"],
    "year": [1995, 2009, 2008, np.nan]
})

raw_movies
#>      film    year
#> 0 Toy Story 1995.0
#> 1      Up   2009.0
#> 2  WALL-E  2008.0
#> 3 Elemental    NaN

md = MoviesDataset(raw_movies)
```

```
md.cleaned
#>      film      year
#> 0 Toy Story 1995.0
#> 1      Up 2009.0
#> 2 WALL-E 2008.0
```

The `MoviesDataset` class has been constructed by *composition* rather than *inheritance*. Composition is an OOP design approach where instances of one class refer to instances of another. So, in a sense our `MoviesDataset` instances will contain `DataFrame`s, but will not *be* `DataFrame`s.

Composition is how we specify *has-a* relationships in OOP: a car has an engine, so a `Car` object should contain an `Engine` object.

5.2 Static methods

The `clean` method in our `MoviesDataset` class could be rewritten as a stand-alone function. It removes any row that has a missing value from the raw `DataFrame` and returns a new `DataFrame`.¹

If we did make it a function outside of the `MoviesDataset` class, it might look as follows:

```
def clean_movies(raw_movies):
    return raw_movies.dropna()

class MoviesDataset():
    def __init__(self, raw):
        # 'raw' and 'cleaned' are pandas DataFrames
        self.raw = raw
        self.cleaned = clean_movies(self.raw)
```

Note that we do not pass a `MoviesDataset` instance into `clean_movies()`, so the function can work independently of the `MoviesDataset` class and its instances. But we have lost some encapsulation: `clean_movies()` was defined for the purpose of working with `MoviesDataset` objects but it is no longer associated with that class.

Static methods provide a way to attach a function that behaves independently of a class or its instances to that class. You might view these as helper functions for the class.² In the class definition, to declare a method as a static method, we use a new decorator: `@staticmethod`.

```
class MoviesDataset():
    def __init__(self, raw):
        # 'raw' and 'cleaned' are pandas DataFrames
        self.raw = raw
        self.cleaned = self.clean(self.raw)

    @staticmethod
    def clean(raw):
        return raw.dropna()
```

¹ Admittedly, any real-world data-cleaning step would be a bit more extensive.

² If a static method performs a function that isn't directly relevant to the class it is attached to, or may be useful for other classes, it may make more sense to define it as a standalone function.

```
md = MoviesDataset(raw_movies)
```

```
md.cleaned
```

```
#>      film    year
#> 0 Toy Story 1995.0
#> 1      Up   2009.0
#> 2 WALL-E   2008.0
```

In fact, because our new static method is independent of `MoviesDataset` instances, we can access it as if it was a standalone function:

```
MoviesDataset.clean(raw_movies)
```

```
#>      film    year
#> 0 Toy Story 1995.0
#> 1      Up   2009.0
#> 2 WALL-E   2008.0
```

5.3 Class methods

Unlike static methods, *class methods* have access to the class (but not to an instance). As such, you can call any other class or static method from inside a class method - including the class constructor.

Suppose we wanted to construct a `MoviesDataset` instance from the contents of a .csv file.³ At present, we'd need to create a `DataFrame` from the file, and then pass that into the `MoviesDataset` constructor.

```
table = pd.read_csv("movies.csv")
new_movies = MoviesDataset(table)
```

Can we modify the class to make this process a little bit easier? One way would be to add a `from_csv()` class method to `MoviesDataset`.

Similarly to the `self` parameter in an instance method, class methods have a `cls` parameter. The `cls` parameter allows access to the class and its methods.⁴ Defining class methods requires the `@classmethod` decorator.

```
class MoviesDataset():
    def __init__(self, raw):
        # 'raw' and 'cleaned' are pandas DataFrames
        self.raw = raw
        self.cleaned = self.clean(self.raw)

    @classmethod
    def from_csv(cls, path):
        table = pd.read_csv(path)
        # Call the constructor for the current class
        return cls(table)

    @staticmethod
    def clean(raw):
        return raw.dropna()
```

³ We couldn't use an instance method for this purpose, because we would then need a class instance before we could make our class instances.

⁴ The parameter is called `cls` rather than `class`, because `class` is already a keyword in Python.

We can now import a movies dataset from a local file.

```
md = MoviesDataset.from_csv("movies.csv")
md.cleaned
#>      Unnamed: 0      film      year
#> 0          0  Toy Story  1995.0
#> 1          1        Up    2009.0
#> 2          2    WALL-E    2008.0
```

Did the syntax seem a little weird? Why should `cls(table)` create a `MoviesDataset`?

The syntax for calling a class method is `TheClass.the_method(... arguments ...)`. Python knows to pass in `TheClass` as the argument for `cls` in the class method (just like it implicitly passes the instance as the `self` argument in an instance method call). So in `MoviesDataset.from_csv("movies.csv")`, the parameters to `from_csv` are `cls = MoviesDataset` and `path = "movies.csv"`. Since `cls` is `MoviesDataset` here, the `cls(table)` call is equivalent to `MoviesDataset(table)`.

Couldn't we have implemented `from_csv` as a static method? For example, as follows:

```
class MoviesDataset():
    # ... snip ...
    @staticmethod
    def from_csv(path):
        table = pd.read_csv(path)
        return MoviesDataset(table)
```

That would work ... but it would be a bit fragile.

As a thought experiment, define a subclass of `MoviesDataset` and call the `from_csv` method on the subclass. It will use the `from_csv` method it inherits from `MoviesDataset`. But the object returned will be a `MoviesDataset`, not an instance of the new subclass.

5.4 Interlude: class attributes

In addition to class methods, you can also attach attributes to a class. These are useful if you need to define values that are used in several methods, or which the user might need. Importantly, class attributes provide a way to share values across all instances of a class.

Suppose we wanted to keep track of the number of instances of a class that have been constructed. A simple class that does this is as follows:

```
class User():
    num_users = 0

    def __init__(self, name):
        self.name = name
        User.num_users = User.num_users + 1
```

Note that class attributes (like `num_users`) are declared outside of the

constructor and we don't use the `self` keyword when declaring them.

Before any instances are constructed, the class attribute is zero:

```
print(User.num_users)
#> 0
```

Then as new instances are created, the attribute is incremented:

```
charlie = User("Charlie")
print(User.num_users)
#> 1

lucy = User("Lucy")
print(User.num_users)
#> 2
```

The class attribute can be accessed using either the namespace for a class (`User.num_users`) or for an instance (`charlie.num_users`).

```
print(charlie.num_users)
#> 2
print(lucy.num_users)
#> 2
```

But, these attributes are mutable and this can lead to some confusion. Suppose an instance modified its copy of a class attribute: should that value affect the value that is accessible by other instances or within the class namespace?

```
charlie.num_users = -1

print(charlie.num_users)
#> -1
print(lucy.num_users)
#> 2
print(User.num_users)
#> 2
```

By calling `charlie.num_users = -1`, we actually defined an *instance* attribute on `charlie`. Now whenever we refer to `charlie.num_users`, it is that instance attribute that is accessed, rather than the class instance.

The main reason for using class attributes is to share mutable values across all instances of a class. But, in our data science work, this is something that we rarely need to do.

5.5 OOP key idea: composition

Although it was a small part of this chapter, the main concept from object-oriented design that was discussed here is *composition*.

Composition is used throughout OOP. Whenever you append an object to `self`, as is common in our constructors (`self.abc = def`), you are using composition. This allows you to add features to a class from another, without having to inherit from that other class. But the values you pass into a constructor can be decided when a program runs, rather than when

it is designed (unlike inheritance hierarchies), so in many settings using composition provides more flexibility than inheritance.

Object-Oriented Design - Polymorphism

There are four pillars of object-oriented programming. These are encapsulation, abstraction, inheritance (which we have met in previous chapters) and polymorphism.

Finding a good definition for polymorphism, and an explanation of it's importance in OOP is surprisingly difficult. To write your code in a polymorphic way, means to write it such that it can seamlessly work with objects of many (*poly*) different forms (*morph*). We have already developed many of the skills needed to write polymorphic code, so this should be a relatively short chapter.

6.1 A polymorphic function

In the following code, we use the same function `len` to assess the size of a range of different objects:

```
len([1, 4, 3]) # List of numbers
#> 3

len(["a", "b"]) # List of strings
#> 2

len("My string")
#> 9

len({"a", "set"})
#> 2

len({"a": "dictionary"})
#> 1
```

This is a polymorphic function: we are using the same function, and applying it to objects of various different types. Here, we have used different types of collection (lists, strings, sets, dictionaries, etc.).

How is this possible? You might imagine there is a really big function defined somewhere in Python, with logic like:

```
def len(x):
    if isinstance(x, list):
        return do_something_with_lists(x)
    elif isinstance(x, str):
        return do_something_with_strings(x)
    # ... and so on ...
    else:
        return 0
```

Suppose that was the case and we wrote a new class. If we wanted to call `len()` on its instances, we would have to modify that `len()` function so that it can work with our class.

There must be a better way!

The actual implementation of `len()` is considerably simpler and looks something like this:

```
def len(x):
    return x.__len__()
```

Now, `len()` works for any class that implements a `*.__len__()` magic method. For this to work with our newly-written class, we just have to define that magic method.

6.2 Additional polymorphic examples

We saw that `len()` can be used with many collection types (regardless of the values stored in the collection). There are many further examples of polymorphic code throughout Python.

The `sorted()` function, which sorts collections, can work with lists of strings, or lists of numbers.

```
for xs in [
    ["A", "D", "B"],
    [10, 45, 1],
    [-1.5, 2.4, float("-inf"), 0.0]
]:
    print(sorted(xs))
#> ['A', 'B', 'D']
#> [1, 10, 45]
#> [-inf, -1.5, 0.0, 2.4]
```

It can actually sort any class provided a less-than method (`*.__lt__()`) is defined.

```
class Colour():
    colour_score = {
        "red": 0,
        "orange": 1,
        "yellow": 2,
        "green": 3,
        "blue": 4,
        "indigo": 5,
```

```

    "violet": 6
}

def __init__(self, colour):
    self.colour = colour

def __lt__(self, other):
    cols = self.colour_score
    return cols[self.colour] < cols[other.colour]

def __repr__(self):
    return self.colour

sorted(
    [Colour("blue"), Colour("yellow"), Colour("indigo")]
)
#> [yellow, blue, indigo]

```

Another example is the `+` operator, which can work with strings, numbers, or collections of objects.

```

for x, y in [
    (1, 4),
    ("a", "b"),
    (["Snoopy", "Charlie"], ["Lucy"])
]:
    print(x + y)
#> 5
#> ab
#> ['Snoopy', 'Charlie', 'Lucy']

```

6.3 *If it walks like a duck ...*

Python is a dynamically-typed language. That means that the type of object bound to a variable is determined when the program runs. So, the same variable can be bound to a string at one point, and then bound to a number (or some other class instance) later on.

```

a = "ZYX"
a = 987
a = f() # We might not even know
        # .. what this function returns

```

This is in contrast to statically-typed languages where the type of each variable must be specified in code. The equivalent of the first line above would have a type declaration before the variable name (`String a = "ZYX";`) in Java, for example, and there would be no way to assign a number to `a` later in the code.

When running, Python doesn't check that the arguments passed to a function or method are of the 'correct' type, either (though if you pass the wrong kind of object into a function, you may find that Python complains

and crashes). Instead, Python uses an approach called *duck-typing*.

In duck-typing, if you pass an object into a function, and that function tries to access a method (or an attribute) on that object, then that method must exist. That is, “if it walks like a duck, and quacks like a duck, it is a duck”.

```
def mystery(x):
    print(x.quack())

class Duck():
    def quack(self):
        return "Quack!"

class NotADuck():
    def woof(self):
        return "Woof!"

class DuckWhistle():
    def quack(self):
        return "QUUUAAAAAACCKKKKK!"
```

```
mystery(Duck())
#> Quack!
```

```
mystery(NotADuck())
#> AttributeError: 'NotADuck' object has no attribute
#> 'quack'
```

```
mystery(DuckWhistle())
#> QUUUAAAAAACCKKKKK!
```

6.4 Polymorphism and inheritance

One of the places where polymorphism is most powerful is in inheritance hierarchies. If a method (or attribute) is defined on a parent class, then that method will be defined on any child class that derives from it.

Following our example in the previous chapter (but with a slightly different implementation), imagine you were working with a range of different datasets: a movies dataset and an actors dataset. For each of these, you need to read the data into your session and run some data-cleaning steps before you can use it. But the data-cleaning steps may differ between the different types of dataset.

It would be a bit scruffy to have a `clean_movies()` function and a `clean_actors()` function. When you have many, similarly named, functions it's easy to type in the wrong function. This also tends to lead to code that *looks* duplicated but isn't - a setting that usually indicates a missing abstraction somewhere along the line.

```
movies = MoviesDataset()
actors = ActorsDataset()
```

```
movies = clean_movies(movies)
actors = clean_actors(actors)
```

It would be a lot neater to have a `*.clean()` method on each class (as we did for `MoviesDataset` in the previous chapter).

```
class MoviesDataset():
    def __init__(self, filepath):
        self.data = pd.read_csv(filepath)

    def clean(self):
        print("Cleaning the movies dataset")
        # do something with self.data

class ActorsDataset():
    def __init__(self, filepath):
        self.data = pd.read_csv(filepath)

    def clean(self):
        print("Cleaning the actors dataset")
        # do something else with self.data
```

```
movies = MoviesDataset("movies-2021.csv")
actors = ActorsDataset("actors.csv")

movies.clean()
#> Cleaning the movies dataset
actors.clean()
#> Cleaning the actors dataset
```

Now that looks a bit nicer. We don't have to remember the different names for the functions that clean the different types of dataset. But the code doesn't demonstrate polymorphism yet. Remember that polymorphism is being used when the code that you write can work with objects of various different forms. Here, our two classes each implement a `*.clean()` method, and we want to write our code so that it can work generically over any class that implements this method.

```
movies = MoviesDataset("movies-2021.csv")
actors = ActorsDataset("actors.csv")

for dataset in [movies, actors]:
    dataset.clean()
#> Cleaning the movies dataset
#> Cleaning the actors dataset
```

Hopefully you can see how that code could be extended if we had movies datasets for some other years, or if we had a `DirectorsDataset()` or `StudioDataset` class as well.

The polymorphic version of the code only works because there is a `*.clean()` method defined on the two classes. This concordance between

the two classes is entirely implicit. One way to guarantee that a set of classes all expose the same methods (so that we can work polymorphically over their instances) is to have them all derive from a common superclass.

So, here we could ensure that `MoviesDataset` and `ActorsDataset` both derive from some class that has a `*.clean()` method.¹ But it doesn't really make sense to define a `*.clean()` method for a class that only exists as a parent for other classes. So what do we do?

¹ It certainly wouldn't make sense for `MoviesDataset` to derive from `ActorsDataset`, or vice versa.

6.5 Interlude: Know your ABCs (abstract base classes)

Python's "Abstract Base Classes" (ABC) provide a way to define a class that will *never* be instantiated. Weird though it may sound, this is quite useful. Though you can't create objects from an abstract class, you can instantiate from any concrete (non-abstract) subclass of that abstract class.

The abstract class defines an interface for the user, and all of its subclasses must comply with that interface.

Because no instances of an abstract class will exist, we can put unimplemented methods in its definition, these are marked with the `@abstractmethod` decorator. It doesn't matter that they aren't implemented, because no instances of the class can call those unfinished methods. All of these abstract methods must be implemented in a concrete subclass.

Continuing our example, we could define an `AbstractDataset` class that `MoviesDataset` and `ActorsDataset` both derive from. By making `AbstractDataset` an abstract base class we can declare a `*.clean()` method that is unimplemented. By doing so we force any derived classes to implement that method.

```
from abc import ABC, abstractmethod

class AbstractDataset(ABC):
    @abstractmethod
    def clean(self):
        pass
```

First, note that `AbstractDataset` cannot be instantiated:

```
not_a_dataset = AbstractDataset()
#> TypeError: Can't instantiate abstract class
#> AbstractDataset without an implementation for abstract
#> method 'clean'
```

Similarly, if we define a subclass of `AbstractDataset` that does not define a `clean` method, then we can't instantiate from that subclass - all abstract methods must be implemented in a concrete subclass:

```
class Uncleanable(AbstractDataset):
    def __init__(self, data):
        self.data = data

unclean = Uncleanable([1, 3, 4])
#> TypeError: Can't instantiate abstract class Uncleanable
```

```
#> without an implementation for abstract method 'clean'
```

Now, when we define our `MoviesDataset` class² as a subclass of `AbstractDataset` we provide a guarantee that all the methods in `AbstractDataset` are defined. This makes it easier to write polymorphic code.

² The `ActorsDataset` is virtually identical and is omitted.

```
class MoviesDataset(AbstractDataset):
    def __init__(self, filepath):
        self.data = pd.read_csv(filepath)

    def clean(self):
        print("Cleaning the movies dataset")
        # do something with self.data
```

```
movies = MoviesDataset("movies-2021.csv")
movies.clean()
#> Cleaning the movies dataset
movies.data
#>
#> distributor      gross      title
#> 0              Sing 2  Universal Pictures
#> 408398852
#> 1              Encanto  Walt Disney Pictures
#> 256786742
#> 2              Jujutsu Kaisen 0          Toho
#> 196436179
#> 3 The Boss Baby: Family Business  Universal Pictures
#> 146745280
#> 4              PAW Patrol: The Movie  Paramount Pictures
#> 144327731
```

6.6 OOP key idea: polymorphism

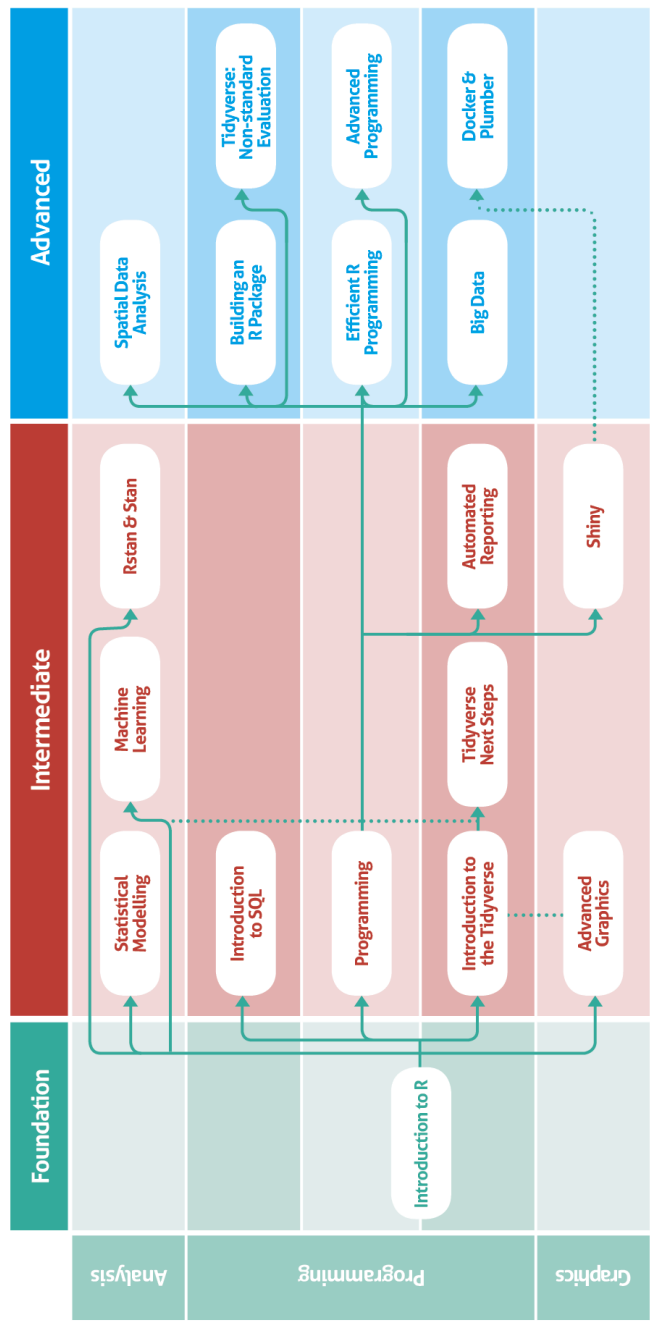
What is the essence of polymorphism? It is about writing your code and constructing your classes so that it makes it easy to work with objects that have different structures. To do this you might write your functions so that they can accept a range of different inputs. Or you might write your classes so that when they have similar behaviour, they also have a similar interface (identical method and attribute names, methods with identical return types).

Those aspects of polymorphism seem like good ideas that aren't directly related to object-oriented programming. They make your code simpler to use, easier to interpret, and harder to misuse. So why is this a key idea for OOP (rather than for programming more generally)? Because classes are so fundamental to OOP, and because inheritance provides guarantees about the interface for a class, if our code needs to use a set of methods from some class instance then we know that code will work with any subclass instance too. So inheritance makes it slightly easier to implement polymorphic code.

Python is a more flexible language than some. For example, in Java, you

have to be precise about the type of object that can be passed into a method as an argument. The compiler will complain if your code attempts to pass the wrong kind of object in. In Python, this doesn't happen. You can write a function that accepts an object as its argument, and provided that the object defines all the methods and attributes required by the function (and they behave appropriately), then it can be used inside the function. The type/class of the object is much less important than its behaviour. This flexibility also makes it easier to write polymorphic code in Python.

Training Course Dependencies





OUR PARTNERS